Angular

What is Angular?

A powerful, open-source web framework by Google for building dynamic, scalable, and interactive single-page applications (SPAs) using TypeScript.

What is Single Page Application?

It is a Modern web app that loads a single HTML page and dynamically updates content using Java Script as the user interacts, avoiding full page reload.

examples: Gmail, Netflix, Spotify, WhatsApp Web …

What is a Library in programming?

Library is a collection of pre-written, reusable code, such as functions, classes, and modules, that developers can use to perform specific tasks without writing the code from scratch.

examples: Axios / Fetch API, jQuery, React Router, React Navigation…

What is a Framework?

A pre-written, foundational structure (code, libraries, tools, rules) that provides a standardized way for developers to build applications quickly, handling common tasks so coders can focus on unique features, enforcing structure, and ensuring efficiency and quality.

examples: Angular, Vue.js, Flutter, Spring …

History of Angular

- **2010** – **AngularJS** released by Google engineer **Miskon Hevery** (JavaScript-based, MVC framework).
- **2012** – Gained popularity for **two-way data binding** and **dynamic web apps**.
- **2014** – Google announced a **complete rewrite** of Angular (called Angular 2).
- **2016** – **Angular 2** officially released, using **TypeScript** for better scalability.
- **2016–Present** – Angular evolves with versions 4, 5… up to **Angular 17** (latest), focusing on:
    - Performance improvements
    - Modern TypeScript features
    - Modular & component-based architecture
- **Purpose:** Build **large, maintainable, and scalable web applications** efficiently.

Features of Angular

Feature	Description
Component-Based Architecture	App is built using reusable, modular components.
TypeScript Support	Uses TypeScript for static typing and safer code.
Two-Way Data Binding	Changes in UI update the model and vice versa automatically.
Dependency Injection (DI)	Makes components modular, testable, and reusable.
Directives	Extend HTML with custom behaviour (e.g., *ngFor, *ngIf).
Routing	Built-in Angular Router for navigation in Single Page Applications (SPA).
Reactive Forms & Validation	Handles complex forms with validation and dynamic control.
HTTP Client	Makes API calls simple with built-in HttpClient module.
Testing Support	Built-in tools for unit and integration testing (Karma, Jasmine).

Advantage and Disadvantage of Angular

Advantages (Good things)	Disadvantages (Bad things)
Good for big applications	Hard to learn at first
Uses TypeScript (fewer bugs)	Too much code for small apps
Everything is built-in	Takes time to understand
Easy to manage large teams	Heavy setup
Strong structure & rules	Less freedom
Good for long-term projects	Not good for quick demos
Powerful forms & validation	App size can be large

Prerequisites for Angular

- Node js
- npm
- angular cli - (npm install -g @angular/cli)

Angular CLI

The Angular CLI is a command-line interface tool which allows you to scaffold, develop, test, deploy, and maintain Angular applications directly from a command shell.

Types of Application best suited for Angular

- Admin Dashboards
- Enterprise portals
- Financial apps
- CRM/ERP systems

Angular Project Structure

File / Folder	Purpose / Description
node modules/	Contains all installed npm packages and Angular libraries
package.json	Lists project dependencies, scripts, and metadata
angular.json	Angular workspace configuration for build and serve
tsconfig.json	TypeScript compiler configuration
tsconfig.app.json	TypeScript configuration specific to the app
README.md	Project documentation
src/	Main source folder of the Angular application
src/main.ts	Entry point; bootstraps the root module
src/index.html	Main HTML file that loads the Angular app
src/styles.css	Global styles for the entire application
src/assets/	Stores static files like images, icons, fonts

src/environments/	Environment-specific configuration files
src/app/	Contains core application logic
app.module.ts	Root module of the application
app.component.ts	Root component logic (TypeScript)
app.component.html	HTML template of the root component
app.component.css	Styles of the root component
app.component.spec.ts	Unit test file for the component
app-routing.module.ts	Defines routes and navigation paths
components/	Stores reusable UI components
services/	Contains services for business logic & API calls

What happens when we run the dev Server

For the dev server, vite and esbuild will be used to generate/create the bundle.(ideally it is named as main.js)

1. The dev server will inject the bundle and the link for global CSS and serve the index.html on the port (**Default :4200**).
2. When we open the browser on this port it will look for entry point code i.e., it will look for the bootstrap function.
3. The bootstrap function will look for root component and find the selector for the root component inside the index.html.
4. Whatever is written inside the root component html file will be displayed inside the <app-root></app-root>

This processing is called bootstrapping angular application (starting).

@angular/platform-browser

It is the library which will provide the necessary environment to run our angular project in the browser. It will help render our components on the UI. And it will make sure the browser is able to properly detect the events, we use in our angular project. Also, this library provides the bootstrap Application , which will help to attach our root component to read DOM

bootstrapApplication

Bootstraps an instance of an Angular application and renders a standalone component as the application's root component.

```
import { bootstrapApplication } from "@angular/platform-browser";
import { App } from "./app/app";

bootstrapApplication(App).catch((error)=>console.log(error)
)
```

What is a Component in Angular?

A **component** is the **basic building block** of an Angular application. It controls a **part of the user interface** and contains **logic, template, and styles.**

Structure of an Angular Component

File	Purpose
component.ts	Contains business logic (TypeScript class)
component.html	Defines UI structure (HTML template)
component.css	Component-specific styles
component.spec.ts	Unit testing file

component.ts

Part	Description
@Component	Decorator that defines metadata
selector	Custom HTML tag name
templateUrl	Path to HTML file
styleUrls	Path to CSS file

```
import { Component } from '@angular/core';

@Component({
  selector: 'app-sample',
  templateUrl: './sample.component.html',
  styleUrls: ['./sample.component.css']
})
export class SampleComponent {

}
```

@Component

Is a Decorator which will help angular to identify the component. It will add metadata to the component ,metadata such as location of the template the style of the component, the selector (custom tag of the component template).

Note:

Name of the component is the class identifier . Selector is the custom tag we will use for the template.

Angular has provided a component name generate the component :

```
ng generate component component-name
```

(or)

```
ng g c component-name
```

Note: What is Standalone Component?

A new feature that allows you to create components without the need for a module (ngmodule).This simplify your application structure and reduce boilerplate code.

What is Data Binding in Angular?

Data Binding in angular is a powerful mechanism for synchronizing data between the component(model) and the template(view), allowing for Dynamic and interactive application.

Why Data Binding is Needed?

- UI updates automatically
- Code becomes clean
- Less DOM manipulation

Types of Data Binding in Angular

1. Interpolation
2. Property Binding
3. Event Binding
4. Two-Way Data Binding

1.Interpolation ({{ }})

Displays data from **component to view.** One-way binding (TS → HTML). Value inside {{ }} comes from TypeScript and shows in HTML.

Important Rules of Interpolation ({{ }})

Rule No.	Rule	Explanation	Example
1	Only expressions allowed	No if, for, while, assignments	{{ price * qty }}
2	One-way data binding	Data flows from TS → HTML only	{{ username }}
3	No HTML tags inside	Interpolation outputs plain text	{{ name }}
4	Not for HTML attributes	Use property binding instead	[src]="imgUrl"
5	No global objects	window, document not allowed	{{ pageTitle }}
6	Avoid heavy logic	Impacts performance	{{ total }}

Example :

```
app.component.ts
------------------------

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {

  companyName = "TechNova";
  trainer = "Michael Johnson";
  course = "Angular Development";
  students = 30;
  price = 4500;
  country = "USA";

  getWelcomeMessage() {
    return "Welcome to the Angular Bootcamp";
  }
}
```

```
app.component.html
------------------------

<h1>{{ companyName }} Training Program</h1>
<p>Trainer Name: {{ trainer }}</p>
<p>Course: {{ course }}</p>
<p>Country: {{ country }}</p>
<p>Total Students Enrolled: {{ students }}</p>
<p>Course Price: ${{ price }}</p>
<p>Discount Price: ${{ price - 500 }}</p>
<p>{{ getWelcomeMessage() }}</p>
<p>Next Batch Expected Students: {{ students + 15 }}</p>
```

What is Property Binding?

Property Binding is used to **bind data from the component (.ts) to an HTML element's property** (TS → HTML). It uses **square brackets [ ]**

Property binding in angular enables you to set value for the property of HTML elements to angular components and more .

Use property binding to dynamically set values for properties and attributes.

Note: We shoulden use property binding for those attributes which expect the value hidden, required. read-only, content Editable.

Example :

```
app.component.ts
------------------------

import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {

  userName = "David Smith";
  profileImage = "https://i.pravatar.cc/200?img=15";
  isButtonDisabled = false;
  boxWidth = "200px";
  boxColor = "lightblue";
  inputPlaceholder = "Enter your email";
}
```

```
app.component.html
------------------------

<h2>User Profile</h2>
<!-- Image Property Binding -->
<img [src]="profileImage" width="120">
<br><br>
<!-- Input Value Binding -->
<input [value]="userName">
<br><br>
<!-- Placeholder Property Binding -->
<input type="email" [placeholder]="inputPlaceholder">
<br><br>
<!-- Button Disable Binding -->
<button [disabled]="isButtonDisabled">Submit</button>
<br><br>
<!-- Style Property Binding -->
<div [style.backgroundColor]="boxColor"
     [style.width]="boxWidth"
     style="height:100px">
  Profile Card
</div>
```

What is Event Binding?

Event Binding is used to **send data or actions from the HTML (view) to the TypeScript file (component)** (HTML → TS) when a user **performs an action** like:

- Click, Key press, Mouse over, Form submits

It uses **round brackets ( ).**

- In Angular, all the events are executed in Bubbling phase, Angular does not provide any support to capture phase. if we want to use capture phase we have to write our custom event handling logic. We can do this with the help of “Custom Directives”.
- If we want to access the event object for an element, we can use the $event, which help us to use the browser’s native event object.
- Angular makes sure that the events work properly in each and every browser with the help of @angular/platform-browser.

Example :

```
app.component.ts
------------------------
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {

  userName = "Emma Watson";
  message = "";
  inputValue = "";

  showAlert() {
    alert("Welcome to Angular Event Binding!");
  }

  showMessage() {
    this.message = "Button Clicked Successfully!";
  }

  getInputValue(event:any) {
    this.inputValue = event.target.value;
  }

  mouseHover() {
    this.message = "Mouse is over the box";
  }
}
```

```
app.component.html
-----------------------

<h2>Event Binding Example</h2>
<!-- Click Event -->
<button (click)="showAlert()">Click Me</button>
<br><br>
<!-- Click Event updating message -->
<button (click)="showMessage()">Show Message</button>
<p>{{ message }}</p>
<br>
<!-- Input Event -->
<input type="text" (input)="getInputValue($event)" placeholder="Type
something">
<p>You typed: {{ inputValue }}</p>
<br>
<!-- Mouseover Event -->
<div (mouseover)="mouseHover()"
     style="width:200px;height:100px;background-color:lightgray;">
  Hover over this box
</div>
```

Interpolation vs Property Binding vs Event Binding

Feature	Interpolation	Property Binding	Event Binding
Purpose	Display data from component to HTML	Bind component data to an HTML element property	Capture user actions and send data from HTML to component
Data Flow Direction	Component → View	Component → View	View → Component
Syntax	{{ expression }}	[property]="expression"	(event)="method()"
Used For	Showing text values	Setting element properties like src, disabled, value	Handling user actions like click, input, submit
Works Inside Attributes?	No	Yes	No

Can Update Data?	No (read-only)	No (one-way)	Yes
Common HTML Examples	Text, headings, paragraphs	src, href, disabled, checked	click, key up, change
Example Code	{{ name }}	<img [src]="imageUrl">	<button (click)="submit()">Submit</button>
Real-World Use Case	Display username	Disable button based on condition	Submit form / add to cart
Best Practice	Use only for text	Use for attributes & DOM properties	Use for user interaction

Two-Way Data Binding

Two-way data binding means **Data flows from Component → UI and UI → Component automatically.**

- If the **variable changes**, the **UI updates.**
- If the **user updates the UI**, the **variable updates.**

A mechanism that automatically Synchronizes data between the components data model and the templates user interface.

Two-way data binding is syntactic sugar for property binding and event binding working together to synchronize data between the component and the view.

Note:

Syntactic sugar is syntax designed to make code easier to read and write while performing the same operation behind the scenes.

ngModel in Two-Way Data Binding

ngModel is the **directive used in Angular to achieve two-way data binding** between the **component (TS)** and the **template (HTML)**.

ngModel is the syntactic sugar for event binding and property binding combined. we need import ngModel from library @angular/forms.

What does ngModel do?

ngModel keeps the input value and the component variable in sync automatically.

- Component → View (property binding).
- View → Component (event binding).
- The [( )] is called **banana in a box.**
- import:[FormsModule] then only ngModule will work.

Syntax

[(ngModel)]="variableName"

Example :

```
app.component.ts
------------------------
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  templateUrl: './app.component.html'
})
export class AppComponent {

  userName = "";
  message = "";

  submitForm(){
    this.message = "Welcome " + this.userName;
  }

  clearInput(){
    this.userName = "";
    this.message = "";
  }
}
```

```
app.component.html
------------------------
<h2>User Registration</h2>
<!-- Two-way binding -->
<input type="text"
       placeholder="Enter your name"
       [(ngModel)]="userName">
<br><br>
<!-- Click event -->
<button (click)="submitForm()">Submit</button>
<!-- Double click event -->
<button (dblclick)="clearInput()">Clear</button>
<p>{{message}}</p>
```

Directives

A directive is a class that allows us to change the appearance, behaviour, or structure of DOM elements in Angular.

- Directives **control HTML**
- They **add logic to the template.**
- Directives run at runtime on HTML elements. example: *ngIf , *ngFor , ngClass , ngStyle.

Why Directives Are Needed?

HTML alone is **static**. Directives make it **dynamic**:

✓ Show / hide elements

✓ Add styles dynamically

✓ Loop data

✓ Handle conditions

✓ Reuse UI behaviour

Types of Directives in Angular

Angular has **3 main types**:

1. **Component Directives**
2. **Structural Directives**
3. **Attribute Directives**
4. **Custom Directives**

Component Directive

A **Component Directive** (or simply *Component*) is a special type of directive that:

- Has its **own template (HTML)**
- Has **logic (TypeScript)**
- Has **styles (CSS)**

Note : Every Angular component is a directive, but **not every directive is a component**.

- @Component is a directive because it can control the template and show the template in the UI. It can also control the appearance of the component, with the help of styles.
- This is the reason why is @Component decorator is also known as Directive.

Attribute Directive

An **Attribute Directive** is a directive that: - **Changes the appearance or behaviour** of an existing DOM element

- **Does NOT add or remove elements**
- Works on the **same element it is applied to**

It modifies **how an element looks or behaves**, not **what exists**.

Common Built-in Attribute Directives

Directive	Purpose
ngClass	Add / remove CSS classes
ngStyle	Apply inline styles
ngModel	Two-way data binding

Structural Directive

A **Structural Directive** is a directive that:

- **Changes the DOM structure**
- **Adds, removes, or manipulates elements**
- Controls **what exists in the DOM**

It decides **whether an element should exist or not**.

Directive	Purpose
***ngIf**	Conditionally add/remove elements
***ngFor**	Render lists
***ngSwitch**	Conditional rendering

What are @if, @for, @switch, @empty?

These (@if, @for, @switch, @empty)are **Agular's NEW control-flow syntax** (introduced in **Angular 17+**). They **replace structural directives** like (*ngIf , *ngFor , *ngSwitch).

They work at **compile time**, not runtime like *ngIf.

Block	Replaces	Purpose
@if	*ngIf	Conditional rendering
@else / @else if	ng-template	Alternate conditions
@for	*ngFor	Loop rendering
track	trackBy	Performance optimization
@empty	Extra *ngIf	Handles empty lists
@switch	*ngSwitch	Multiple condition rendering
@case	*ngSwitchCase	Match case
@default	*ngSwitchDefault	Default case

Note:

Structural directives and control flow blocks both modify DOM structure, but structural directives work at runtime using directives, while control flow blocks work at compile time using the Angular compiler.

In an element , we can give only one structural directive ,we can't give more than one structural directive to an element ,Angular does not support that.

@for :

- A built-in template syntax in Angular(v17) used to repeatedly render HTML content for each item in a collection.
- It is a more concise and performance, optimized alternative to the older *ngFor directive, requiring no extra imports.

@for track :

- The track expression is a mandatory part of the @for syntax used to uniquely identify each item in the list for optimized DOM updates.

Why track is better?

- Angular identifies items using a **unique key**
- Only changed items are updated
- Better **performance**
- Prevents **DOM flickering**
- Essential for: Large lists , Forms , Animations.

@empty :

The @empty block in Angular is part of the new built-in control flow syntax and is used to declaratively render content when the collection in an @for loop is empty. It replaces the more verbose method of using @ngIf to check the arrays length.

- The @empty block must immediately follow an @for block with your template.

Contextual Variables in Angular (@for, *ngFor)

Contextual variables are special variables Angular provides **inside loops** to give extra information about the current iteration (index, first item, last item, etc.).

Variable	Meaning
$index	Current index (0-based)
$count	Total number of items
$first	true if first item
$last	true if last item
$even	true if index is even
$odd	true if index is odd

Why Contextual Variables are Useful

- Show serial numbers
- Apply alternate row colors
- Disable first/last buttons
- Conditional styling

Note:

@for uses $ prefix for contextual variables.

ngFor uses plain names.

Example :

```
app.component.html
------------------------
<h2>Angular Directives Demo</h2>
<!-- @if Directive -->
@if(isLoggedIn){
  <p>Welcome to Angular Training</p>
}
<hr>
<h3>Student List</h3>
<ul>
@for(let std of students; track $index){
  <li
    [ngClass]="{'highlight': $first}"
    [ngStyle]="{'color': $even ? 'blue' : 'green'}"
  >
    {{ $index + 1 }} - {{std}}
    @if($first){
      <span> (First Student)</span>
    }
    @if($last){
      <span> (Last Student)</span>
    }
    @if($even){
      <span> (Even Index)</span>
    }
    @if($odd){
      <span> (Odd Index)</span>
    }
  </li>
}
</ul>
<hr>
<!-- ngModel -->
<input type="text" [(ngModel)]="userName" placeholder="Enter your name">
<p>Hello {{userName}}</p>
```

```
app.component.ts
-------------------------
import { FormsModule } from '@angular/forms';
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  imports:[FormsModule],
  templateUrl: './app.component.html'
})
export class AppComponent {

  isLoggedIn = true;

  students = [
    "James Smith",
    "Emma Watson",
    "Lucas Brown",
    "Olivia Taylor"
  ];

  userName = "";
  color = "blue";
}
```

Custom Directive

A **Custom Directive** in Angular is a directive **created by the developer to add custom behavior to HTML elements**.

It allows you to **change the appearance or behavior of an element using your own logic**.

Create Directive using CLI

```
ng generate directive highlight
```

Or

```
ng g d highlight
```

Example :

```
highlight.directive.ts
-----------------------
import { Directive, ElementRef } from '@angular/core';

@Directive({
  selector: '[appHighlight]'
})
export class HighlightDirective {

  constructor(private el: ElementRef) {
    this.el.nativeElement.style.backgroundColor = "yellow";
  }

}
```

```
app.component.html
-----------------------
<h2>Angular Custom Directive Example</h2>

<p appHighlight>
  This text is highlighted using custom directive
</p>
```

@Directive

@Directive is a **decorator used to create a custom directive** in Angular.

It tells Angular that the **class is a directive and can modify HTML elements**.

Syntax

```
@Directive({
  selector: '[directiveName]'
})
```

ElementRef

ElementRef is used to **access the DOM element directly**. It allows the directive to **change styles, content, or behaviour of an HTML element**.

Syntax

```
constructor(private el: ElementRef){}
```

el refers to the **HTML element where the directive is applied**.

@HostListener

@HostListener in Angular is used to **listen to events on the host element** (the element where the directive is applied).

It allows the directive to **respond to events like click, mouseover, keypress, etc.**

Component Communication in Angular

What is Component Communication?

Component Communication is the process of sharing data between components in an Angular application.

Angular applications are built using multiple components, and these components often need to send or receive data from each other.

Why Do We Need Component Communication?

1. **To share data between components**
   Example: Parent component sends user data to a child component.
2. **To trigger actions in another component**
   Example: A child component button clicks updates data in the parent component.
3. **To build modular applications**
   Angular apps are divided into small reusable components, so communication is necessary.
4. **To manage UI interactions**
   Example: Selecting a product in a list should show details in another component.
5. **To maintain clean and organized code**
   Instead of putting everything in one component, we divide logic into multiple components.
6. **To support reusable components**
   Components like navbar, card, form, and tables can receive different data using communication.

Types of Component Communication

Angular mainly supports three types:

Type	Decorator	Direction
Parent → Child	@Input()	Data passing
Child → Parent	@Output()	Event sending
Two Way	@Input() + @Output()	Data sharing

@Input() in Angular

@Input() is a decorator used to pass data from a Parent Component to a Child Component.It allows the child component to receive data from the parent.

Data Flow

Parent Component → Child Component

Syntax

@Input() propertyName : datatype;

Example

```
app-parent-component.ts
---------------------------
import { Component } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  templateUrl: './parent.component.html'
})
export class ParentComponent {

  car = "BMW";

}
```

```
app-parent-component.html
---------------------------
<h1>Parent Component</h1>
<app-child [carName]="car"></app-child>
```

Child-component

```
app-child-component.ts
---------------------------
import { Component, Input } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  templateUrl: './child.component.html'
})
export class ChildComponent {

  @Input() carName!: string;

}
```

```
app-child-component.html
---------------------------
<h2>Selected Car: {{carName}}</h2>
```

@Output() in Angular

@Output() is a decorator used to send data or events from a Child Component to a Parent Component. It works together with EventEmitter.

In simple words: @Output() allows a child component to notify or send data to the parent component.

Why do we use @Output()?

1. To send data from child to parent
2. To trigger actions in parent component
3. To handle events like button click, delete, update
4. To communicate user actions from child UI

Syntax of @Output()

@Output() eventName = new EventEmitter<DataType>();

What is EventEmitter?

EventEmitter is a class in Angular used to send data or events from a Child Component to a Parent Component. EventEmitter is used to emit (send) events from the child component to the parent component.

What is emit()?

emit() is a method of EventEmitter used to send data or trigger an event from a Child Component to a Parent Component.

Why do we use emit()?

1. Send data from **child → parent**
2. Trigger an **event in the parent component**
3. Inform the parent when **something happens in the child**
4. Pass values like **form data, selected item, button action**

Syntax of emit()

eventName.emit(value)

Example:

Child Component

```
app-child-component.ts
----------------------------
import { Component, Output, EventEmitter } from '@angular/core';

@Component({
  selector: 'app-child',
  standalone: true,
  templateUrl: './child.component.html'
})
export class ChildComponent {

  @Output() carEvent = new EventEmitter<string>();

  sendCar(){
    this.carEvent.emit("BMW M5");
  }
}
```

```
app-child-component.html
----------------------------
<h2>Child Component</h2>
<button (click)="sendCar()">
Select Car
</button>
```

Parent Component

```
app-child-component.ts
---------------------------
import { Component } from '@angular/core';
import { ChildComponent } from './child.component';

@Component({
  selector: 'app-parent',
  standalone: true,
  imports: [ChildComponent],
  templateUrl: './parent.component.html'
})
export class ParentComponent {

  selectedCar = "";

  receiveCar(car:string){
    this.selectedCar = car;
  }

}
```

```
app-child-component.html
---------------------------
<h1>Parent Component</h1>

<app-child (carEvent)="receiveCar($event)"></app-child>

<h2>Selected Car: {{selectedCar}}</h2>
```

What is $event?

$event is a special variable in Angular templates that contains the data sent from an event. In component communication: $event is used to receive the value emitted by the child component using emit().

Why do we use $event?

1. Receive data sent from child component
2. Capture event values in parent component
3. Pass emitted data to a parent method

Difference between @Input(), @Output(), EventEmitter, and $event

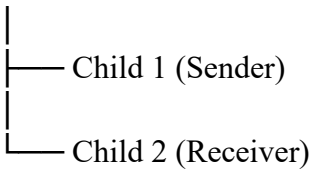
Concept	Purpose	Used In	Direction
@Input()	Receive data	Child component	Parent → Child
@Output()	Create event	Child component	Child → Parent
EventEmitter	Emit event	Child component	Child → Parent
$event	Receive emitted value	Parent template	Child → Parent

Sibling Component Communication in Angular

Sibling Component Communication is the process of sharing data between two child components that belong to the same parent component. These components **cannot communicate directly**, so they communicate **through the parent component**.

Structure

Parent Component



Flow:

Child 1 → Parent → Child 2

Why Do We Need Sibling Communication?

1. To share data between two child components
2. To update UI in another component
3. To handle selection or interaction from another component
4. To maintain component separation and modular design

How Sibling Communication Works

It uses both @Output() and @Input().

Steps:

1. First child sends data using @Output()
2. Parent receives the data
3. Parent passes data to second child using @Input()

Key Points

- Sibling components cannot communicate directly
- Communication happens through parent
- Uses @Output() and @Input()

Pipes in Angular

Pipes are used to transform or format data in the template before displaying it to the user. Pipes take input data and transform it into a desired format in the HTML template.

Example transformations:

- Text → Uppercase
- Date → Formatted date
- Number → Currency

Why Do We Use Pipes?

1. To format data easily in the template
2. To avoid writing formatting logic in the component
3. To make templates cleaner and readable
4. To reuse data transformation logic

Pipe Syntax

{{ value | pipeName }}

Built-in Pipes in Angular

Angular provides several **built-in pipes** that help **transform or format data directly in the template** without writing extra logic in the component.

General syntax:

{{ value | pipeName : parameter1 : parameter2 }}

- value → original data
- | → pipe operator
- pipeName → built-in pipe
- : → used to pass parameters

Built-in Pipes Table

Pipe Name	Purpose	Parameters	Example	Output
uppercase	Converts text to uppercase	No parameters	{{ name | uppercase }}	ALEXANDER SMITH
lowercase	Converts text to lowercase	No parameters	{{ name | lowercase }}	alexander smith
titlecase	Capitalizes first letter of each word	No parameters	{{ name | titlecase }}	Alexander Smith
date	Formats date values	format, timezone, locale	{{ joinDate | date:'dd/MM/yyyy' }}	15/03/2026
currency	Formats number as currency	currencyCode, display, digitsInfo, locale	{{ salary | currency:'USD' }}	$5,000.00
percent	Converts number to percentage	digitsInfo, locale	{{ progress | percent }}	75%
number	Formats numbers with decimal places	digitsInfo, locale	{{ score | number:'1.2-2' }}	95.46
slice	Extracts part of string/array	start, end	{{ name | slice:0:4 }}	Alex
json	Converts object to JSON format	No parameters	{{ student | json }}	{ "name":"Alexander Smith" }
async	Displays data from Observable or Promise automatically	No parameters	{{ data$ | async }}	Displays emitted value

What is Pipe Chaining?

Pipe Chaining means using multiple pipes together on the same value to transform the data step by step.

The output of the first pipe becomes the input of the next pipe.

Syntax

{{ value | pipe1 | pipe2 | pipe3 }}

Explanation:

Part	Meaning
value	Original data
pipe1	First transformation
pipe2	Second transformation
pipe3	Third transformation

Important Points

- Multiple pipes can be used **in one expression**.
- Pipes are executed **from left to right**.
- Output of **one pipe becomes input of the next pipe**.

Example :

```
app-pipes-component.ts
----------------------------
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { of } from 'rxjs';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule],
  templateUrl: './app.component.html'
})
export class AppComponent {

  employeeName = "alexander smith";
  company = "GLOBAL TECH SOLUTIONS";
  joiningDate = new Date();
  salary = 5000;
  performance = 0.82;
  rating = 4.5678;

  employee = {
    name: "Alexander Smith",
    country: "USA",
    department: "Software"
  };

  skills = ["Angular", "JavaScript", "TypeScript"];

  asyncMessage$ = of("Employee data loaded successfully");
}
```

```
app-pipes-component.html
----------------------------
<section style="font-family:Arial; padding:20px">
<h1>Employee Profile Dashboard</h1>
<hr>
<h2>Text Pipes</h2>
<p>Original Name: {{ employeeName }}</p>
<p>Uppercase: {{ employeeName | uppercase }}</p>
<p>Lowercase: {{ company | lowercase }}</p>
<p>Title Case: {{ employeeName | titlecase }}</p>
<hr>
<h2>Date Pipe</h2>
<p>Joining Date Default: {{ joiningDate | date }}</p>
<p>Custom Format: {{ joiningDate | date:'dd/MM/yyyy' }}</p>
<p>Full Date (Pipe Chaining): {{ joiningDate | date:'fullDate' | uppercase }}
</p>
<hr>
<h2>Currency Pipe</h2>
<p>Salary USD: {{ salary | currency:'USD' }}</p>
<p>Salary INR: {{ salary | currency:'INR' }}</p>
<hr>
<h2>Percent Pipe</h2>
<p>Performance: {{ performance | percent }}</p>
<p>Performance with digits: {{ performance | percent:'1.1-2' }}</p>
<hr>
<h2>Number Pipe</h2>
<p>Rating: {{ rating | number:'1.2-2' }}</p>
<hr>
<h2>Slice Pipe</h2>
<p>Short Name: {{ employeeName | slice:0:5 }}</p>
<p>First Skill: {{ skills | slice:0:1 }}</p>
<hr>
<h2>JSON Pipe</h2>
<pre>{{ employee | json }}</pre>
<hr>
<h2>Async Pipe</h2>
<p>{{ asyncMessage$ | async }}</p>
<hr>
<h2>Pipe Chaining Example</h2>
<p>
Employee Display Name:
{{ employeeName | uppercase | slice:0:8 }}
</p>
</section>
```

Custom Pipe in Angular

A Custom Pipe is a pipe created by the developer to transform data in a way that Angular's built-in pipes cannot. A custom pipe is used to create your own data transformation logic and use it in the template.

Example situations:

- Reverse a string
- Mask email or phone number
- Calculate tax on salary
- Convert text format

Why Do We Use Custom Pipes?

1. To create **custom data transformations**
2. When built-in pipes are **not sufficient**
3. To keep **templates clean and readable**
4. To **reuse transformation logic** across components

CLI Command to Create a Pipe

ng generate pipe pipe-name (or) ng g p pipe-name

Example

Custom pipe:

```
app-pipes-component.ts
---------------------------
import { Pipe, PipeTransform } from '@angular/core';

@Pipe({
  name: 'discount'
})
export class DiscountPipe implements PipeTransform {

  transform(price:number, discount:number){

    return price - (price * discount / 100);

  }
}
```

```
app-component.ts
----------------------------
import { Component } from '@angular/core';
import { CommonModule } from '@angular/common';
import { DiscountPipe } from './discount.pipe';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [CommonModule, DiscountPipe],
  templateUrl: './app.component.html'
})
export class AppComponent {

  productName = "Laptop";

  price = 50000;

  discountPercent = 20;

}
```

```
app-component.html
----------------------------
<section style="padding:20px;font-family:Arial">
<h2>Product Offer</h2>
<p>Product Name: {{productName}}</p>
<p>Original Price: ₹{{price}}</p>
<p>Discount: {{discountPercent}}%</p>
<hr>
<h3>Final Price: ₹{{price | discount:discountPercent}}</h3
</section>
```

Template Reference Variable in Angular

A Template Reference Variable is used to reference an HTML element, Angular directive, or component inside the template. A template reference variable allows you to access an element or its value directly in the HTML template.

It is declared using the **# symbol**.

Syntax :

#variableName

Template Reference Variable Uses

Use Case	Example
Access input value	#name
Call component method	(click)="save(name.value)"
Form validation	#user="ngModel"
Access DOM element	#video
Control UI	focus, clear input

Example:

```
app-component.ts
----------------------
import { Component } from '@angular/core';

@Component({
  selector: 'app-root',
  standalone: true,
  templateUrl: './app.component.html'
})
export class AppComponent {

  searchProduct(product: string) {
    alert("Searching for: " + product);
  }
}
```

```
app-component.html
----------------------
<section style="padding:20px;font-family:Arial">

<h2>Product Search</h2>

<input
type="text"
placeholder="Enter product name"
#searchBox>

<button (click)="searchProduct(searchBox.value)">
Search
</button>
</section>
```

Template Reference vs Two-Way Binding

Feature	Template Reference	Two-way Binding
Syntax	#name	[(ngModel)]
Usage	Direct DOM reference	Data binding
Data Flow	Template → Component	Component ↔ Template

Forms in Angular

What are Forms in Angular

- Forms are used to **collect user input** in web applications.
- They allow users to enter data like **name, email, password, phone number, address, etc.**
- Angular forms help to **manage, validate, and submit user data** to the application or server.
- Forms are widely used in **login pages, registration forms, job applications, contact forms, and payment pages**.

Purpose of Angular Forms

- Collect user information.
- Validate the entered data.
- Track user interaction with form fields.
- Submit data to backend or API.
- Display error messages when data is invalid

Types of Forms in Angular

Angular provides **two main types of forms**:

Template Driven Forms

- Form logic is mostly written in **HTML template**.
- Uses directives like **ngForm** and **ngModel**.
- Simple and easy to implement.
- Best for **small or simple forms**.

Reactive Forms

- Form logic is written in **TypeScript component**.
- Uses classes like **FormGroup**, **FormControl**, and **FormArray**.
- More powerful and scalable.
- Best for **large and complex forms**.

Angular Form Validation

Angular provides built-in validators such as:

- **required** – field must not be empty
- **minlength** – minimum number of characters
- **maxlength** – maximum number of characters
- **pattern** – validate using regular expression
- **email** – check valid email format

Angular Form States

Angular tracks the state of form controls:

- **Valid** – input is correct
- **Invalid** – input is incorrect
- **Touched** – user interacted with the field
- **Untouched** – user has not interacted
- **Dirty** – field value has changed
- **Pristine** – field value has not changed

Real-world Uses of Angular Forms

- Login systems
- User registration forms
- Job application portals
- Contact forms
- Payment forms

Template Driven Forms in Angular

Template Driven Forms are a type of Angular form where **most of the form logic is written inside the HTML template instead of the TypeScript component**. Angular automatically creates and manages the form model using directives such as **ngForm** and **ngModel**. This approach is easier for beginners because it relies on **HTML templates and two-way data binding** to capture user input and perform validation. Template driven forms are commonly used in small or simple applications like login forms, registration forms, and contact forms where complex logic is not required.

Features of Template Driven Forms

- Form logic mainly handled in **HTML template**
- Uses **ngModel** for two-way data binding
- Uses **ngForm** to manage the form
- Supports **built-in validation**
- Angular automatically **creates the form model**
- Less TypeScript code required
- Easy to learn and implement

When to Use Template Driven Forms

- Small or simple forms
- Forms with **basic validation**
- Applications where **form structure is simple**
- Beginner level Angular projects
- When developers prefer **HTML-based form logic**

What is Required to Use Template Driven Forms

Requirement	Purpose
FormsModule	Enables template driven forms
ngModel	Binds input field data
ngForm	Controls the form
name attribute	Identifies form control

Example

```
app-component.ts
------------------------
import { Component } from '@angular/core';
import { FormsModule } from '@angular/forms';

@Component({
  selector: 'app-root',
  standalone: true,
  imports: [FormsModule],
  templateUrl: './app.component.html'
})
export class AppComponent {

  applications:any[] = [];

  submitForm(form:any){

    this.applications.push(form.value);

    form.reset();

  }
}
```

```
app-component.html
------------------------
<section style="padding:20px;font-family:Arial">
<h2>Job Application Form</h2>
<form #jobForm="ngForm" (ngSubmit)="submitForm(jobForm)">
<!-- NAME -->
<label>Name</label><br>
<input type="text" name="name" ngModel required#name="ngModel">
@if(name.invalid && name.touched){
<p>Name is required</p>
}
<br><br>
<!-- EMAIL -->
<label>Email</label><br>
<input type="email" name="email" ngModel required #email="ngModel">
@if(email.invalid && email.touched){
<p>Enter valid email</p>
}
```

```
app-component.html
-----------------------
<br><br>
<!-- PASSWORD -->
<label>Password</label><br>
<input type="password" name="password" ngModel required minlength="6"
#password="ngModel">
@if(password.invalid && password.touched){
<p>Password must be at least 6 characters</p>
}
<br><br>
<!-- GENDER -->
<label>Gender</label><br>
<input type="radio" name="gender" value="Male" ngModel> Male
<input type="radio" name="gender" value="Female" ngModel> Female
<br><br>
<!-- SKILLS -->
<label>Skills</label><br>
<input type="checkbox" name="angular" ngModel> Angular
<input type="checkbox" name="react" ngModel> React
<input type="checkbox" name="node" ngModel> NodeJS
<br><br>
<!-- CITY -->
<label>City</label><br>
<select name="city" ngModel>
<option value="">Select City</option>
<option value="Chennai">Chennai</option>
<option value="Bangalore">Bangalore</option>
<option value="Hyderabad">Hyderabad</option>
</select>
<br><br>
<button type="submit">
Submit Application
</button>
</form>
<hr>
<h2>Submitted Applications</h2>
<table border="1" cellpadding="8">
<tr>
<th>Name</th>
<th>Email</th>
<th>Gender</th>
<th>City</th>
</tr>
@for(app of applications; track $index){
<tr>
<td>{{app.name}}</td>
<td>{{app.email}}</td>
<td>{{app.gender}}</td>
<td>{{app.city}}</td>
</tr>
}
</table>
</section>
```